IPG Job Manager

So the thing is, right now our grid users rely on Globus GRAM to run their jobs on remote machines across IPG, and honestly it just doesn't cut it anymore. People keep asking for things like being able to stage several sets of files at once, run more than one executable as part of the same submission, and actually look back at what happened with a job after the fact — none of that is really there today. We keep hearing complaints that there's no good history of executions, so when something goes wrong nobody can tell what happened or why.

What I'm picturing is a Job Manager that lets users move and copy files between systems and kick off applications on whatever machine they're authorized to use, and they should be able to do this either through an API or from the command line, whichever fits their workflow. The core idea is a job made up of a main sequence of steps — file operations and execution operations chained together reliably — and then, if they want, an optional cleanup part that fires afterwards to delete temp files and directories whether the job succeeded or blew up halfway through. If something fails partway through the main sequence, I'd expect the system to stop, run the cleanup tied to whatever failed plus the cleanup for everything that already ran, going backwards, so we don't leave junk scattered across machines.

It also needs to play nice with how the grid already handles security — there's a proxy/credential thing tied into the Grid Security Infrastructure that has to be respected, and only authorized users should ever be able to touch the systems on the list the Job Manager manages. And whatever we build, it has to keep some kind of resource list so people can only submit to machines that are actually registered, not just anywhere.

On the people side, we've basically got two kinds of usage. Regular users will connect through the client side — find a server, submit a job, check on it, cancel it, look up old runs, that sort of thing. Then there's the admin side: someone has to install and keep the server itself running, someone

manages the actual hardware and OS underneath it (might be the same person, might not), and there's also whoever takes care of the software repository server. Worth keeping in mind these admin folks need their own way to manage who has access, manage that resource list, browse jobs, and have some kind of log of what admin actions were taken.

A couple of things I want to be upfront about not worrying about right now. Running one application split across two machines at the same time with them talking to each other — co-scheduling, that kind of multi-machine parallel execution — that's interesting but not something we need for this round. Same with building out a full workflow/DAG language with a GUI to draw out job flows, staging files in parallel instead of one at a time, juggling file permissions across different OS types when staging, or any kind of file caching to speed things up — all of that can wait, it's not critical for what we need delivered now. Also heads up, this version won't be a drop-in replacement for whatever the old Job Manager did before, so we'll need to help existing users transition rather than assume their old scripts just keep working.

One more practical thing: whatever we ship needs to actually tell people when something breaks — a real error name, a message, and ideally the reason behind it, plus a way to log errors to a file users can point to themselves. And it should have some sense of timeouts so things don't just hang forever waiting on a response.

One more thing since this is federal software built for NASA's IPG program, the whole thing has to meet Section 508 accessibility requirements, not just some corner of it, so that's not something we can skip just because most of our users are pretty technical.